



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

DOCINSIGHT AI: AN INTELLIGENT DOCUMENT ANALYSIS AND QUESTION ANSWERING SYSTEM

A CAPSTONE PROJECT REPORT

Submitted in partial fulfilment of the requirements

For the course of

DSA0306-NATURAL LANGUAGE PROCESSING

towards the award of the degree of

BACHELOR OF

TECHNOLOGY IN

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING (AI&ML)

Submitted by

G. Aswartha Manidheep (192425089)

Under the Supervision of

Dr. Kumaragurubaran T

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled "DocInsight AI: An Intelligent Document Analysis and Question Answering System" has been carried out by Dhanusha. D (192424189) and S. Padmaja (192424159) under the supervision of Dr. Almas Begum and is submitted in partial fulfilment of the requirements for Module 1 – Intelligent Document Processing of the current semester of the B. Tech Artificial Intelligence and Data Science program at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR	COURSE FACULTY / GUIDE
Dr. Devi T	Dr. Kumaragurubaran T
Professor	Professor
Department of CSE	Department of CSE
SIMATS Engineering	SIMATS Engineering

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105

DECLARATION

I G. Aswartha Manidheep (192425089) of the Department of B. Tech Artificial Intelligence and Machine Learning (AI&ML), SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled "DocInsight AI: An Intelligent Document Analysis and Question Answering System" is the result of our own Bonafide efforts.

To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged.

We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place:

Date:

Signature of the Students with Names

G. Aswartha Manidheep (192425089)

INDIVIDUAL CONTRIBUTION STATEMENT

(Mandatory for all team projects)

Student Name / Register No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
G. Aswartha MAidheep(192425089)	Document corpus collection; PDF/DOCX/scanned-document preprocessing; OCR pipeline setup	OCR and text-extraction module development	Validation of extracted text quality; OCR accuracy checking	Ch. 1, Ch. 3 (document-processing sections)	25%
Vasantha Nidhi D Y (192424137)	Text chunking and cleaning workflow; metadata tagging	Chunking and preprocessing pipeline; chunk-size tuning	Chunk-quality validation; preprocessing test records	Ch. 2, Ch. 3 (chunking/preprocessing sections)	25%
Aswanth N N (192424123)	Embedding and retrieval architecture; vector database integration	Embedding-model integration; FAISS vector-store development	Retrieval-relevance checks; embedding evaluation	Ch. 3 (retrieval design sections), Ch. 4 (results)	25%
Mohammed Jameel A (192425057)	LLM-based answer generation; dashboard integration	Retrieval-augmented generation (RAG) pipeline; dashboard development	Answer-accuracy testing; dashboard testing	Ch. 4 (testing/results), Ch. 5, appendices	25%
TOTAL					100%

ABSTRACT

DocInsight AI: An Intelligent Document Analysis and Question Answering System is aimed at transforming unstructured document collections into a searchable, conversational knowledge source that returns accurate, evidence-grounded answers instead of requiring manual reading. Organizations and individuals routinely accumulate large volumes of PDFs, Word documents, and scanned files whose contents are difficult to search, cross-reference, or summarize using conventional keyword search. To address this challenge, the proposed system develops a Document Intelligence Platform that ingests heterogeneous documents, extracts and cleans their text using OCR and layout-aware parsing, splits the content into semantically coherent chunks, and converts each chunk into dense vector embeddings stored in a vector database. When a user submits a natural-language question, the system retrieves the most relevant chunks through semantic similarity search and passes them, together with the question, to a large language model that generates a grounded answer with source citations through a Retrieval-Augmented Generation (RAG) pipeline. The system is organized into three core modules: Module 1 – Intelligent Document Processing (ingestion, OCR, cleaning, and chunking), Module 2 – Semantic Embedding and Retrieval, and Module 3 – Question Answering and Response Generation, integrated through an interactive dashboard that supports document upload, search, and conversational querying. The expected engineering outcome is a reliable, scalable, and accurate document analysis and question-answering platform that demonstrates practical natural-language processing, information-retrieval, and generative-AI engineering for real-world knowledge-management problems.

Keywords

Document AI; intelligent document processing; natural language processing; retrieval-augmented generation; question answering; semantic search; vector embeddings

TABLE OF CONTENTS

Sl. No.	Title	Page No.
i	CERTIFICATE FROM THE SUPERVISOR / BONAFIDE CERTIFICATE	ii
ii	DECLARATION BY THE CANDIDATE	iii
iii	INDIVIDUAL CONTRIBUTION STATEMENT	iv
iv	ABSTRACT	v
v	LIST OF FIGURES	vii
vi	LIST OF TABLES	viii
vii	LIST OF ABBREVIATIONS / SYMBOLS	ix
1	CHAPTER 1: Introduction and Engineering Problem	1
2	CHAPTER 2: Literature Review and Existing Solutions	4
3	CHAPTER 3: Engineering Design & Trade-offs, Sustainability, Ethics, Safety, Risk & Security	6
4	CHAPTER 4: Implementation, Testing & Results, Project Management	11
5	CHAPTER 5: Conclusion & Reflection	15
	References	20
	Appendices	22

LIST OF FIGURES

Figure No.	Figure Name	Page No.
Figure 1	Sample document corpus used for ingestion and analysis	5
Figure 2	Proposed system architecture of the DocInsight AI platform	8
Figure 3	Output of the DocInsight AI question-answering dashboard	12
Figure 4	Document-processing and chunking outputs	12
Figure 5	Retrieval and answer-generation outputs	13

LIST OF TABLES

Table No.	Table Name	Page No.
Table 1	Stakeholder / User Requirements	6
Table 2	Functional and Non-Functional Requirements	6
Table 3	Constraints and Applicable Standards	7
Table 4	Design Specifications	7
Table 5	Alternative Solutions Evaluation	8
Table 6	Trade-off Analysis	9
Table 7	Sustainability, Ethics, Safety and Security Evidence	9
Table 8	Risk Register	10
Table 9	Tools / Software / Equipment	11
Table 10	Test Plan and Verification	13
Table 11	Results / Key Outcomes	14
Table 12	Objective Achievement	15
Table 13	Project Roles	16
Table 14	Project Budget	16

LIST OF ABBREVIATIONS / SYMBOLS

Abbreviation / Symbol	Description	Unit
AI&DS	Artificial Intelligence and Data Science	—
NLP	Natural Language Processing	—
OCR	Optical Character Recognition	—
RAG	Retrieval-Augmented Generation	—
LLM	Large Language Model	—
QA	Question Answering	—
IDE	Integrated Development Environment	—
RAM	Random Access Memory	GB
n	Number of documents / chunks processed	records

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Organizations, research groups, and individuals routinely accumulate large collections of unstructured documents — PDFs, Word files, scanned reports, and manuals — that contain valuable information but are difficult to search, cross-reference, or summarize. Conventional keyword search treats a document as a bag of words and cannot answer natural-language questions that depend on context, paraphrasing, or information scattered across multiple sections or files.

Recent advances in Natural Language Processing (NLP), transformer-based language models, and dense vector embeddings have made it possible to represent document content semantically rather than lexically. Combined with vector databases and Retrieval-Augmented Generation (RAG), these techniques allow a system to locate the most relevant passages for a query and generate a coherent, grounded natural-language answer instead of returning a list of matching pages.

This project applies Intelligent Document Processing and RAG-based question answering to heterogeneous document collections by extracting and cleaning text (including scanned content via OCR), chunking it into semantically meaningful units, embedding the chunks in a vector space, and using a large language model to answer user questions with citations back to the source documents.

1.2 Need for the Project

- Large document collections are difficult to search manually because relevant information is often distributed across many files and sections.
- Keyword search returns whole documents or pages rather than direct, synthesized answers, forcing users to read through irrelevant content.
- Scanned and image-based documents are not searchable at all without OCR and text-extraction support.
- Relationships and answers that span multiple documents are difficult to assemble manually.
- An integrated document-intelligence platform can substantially reduce the time required to move from raw documents to accurate, cited answers.

Stakeholders affected by this problem include knowledge workers and researchers, students and academic staff, legal and compliance teams, customer-support teams, and any organization that maintains large internal document repositories.

1.3 Problem Statement

The engineering problem is to design and implement an intelligent document-analysis and question-answering system that can ingest heterogeneous document formats, extract and clean text (including scanned content), organize the content into retrievable semantic units, generate accurate answers to natural-language questions with source attribution, and present the results through an interactive dashboard suitable for search, upload, and conversational querying.

The problem involves interacting requirements related to text-extraction accuracy, retrieval relevance, answer correctness and grounding, response latency, scalability to large document sets, and responsible handling of potentially sensitive document content.

1.4 Project Aim

To design and develop an intelligent document analysis and question-answering platform that transforms unstructured document collections into an accurate, searchable, and conversational knowledge source using OCR, semantic embeddings, and retrieval-augmented generation.

1.5 Project Objectives

1. To collect and preprocess a heterogeneous corpus of PDF, DOCX, and scanned documents.
2. To build an Intelligent Document Processing module (Module 1) that extracts, cleans, and OCRs document content and splits it into semantic chunks.
3. To generate dense vector embeddings for document chunks and index them in a vector database for semantic search.
4. To develop a Retrieval-Augmented Generation pipeline that retrieves relevant chunks and generates grounded natural-language answers.
5. To support source-citation and traceability so that every generated answer can be linked back to its originating document and passage.
6. To build an integrated, searchable dashboard supporting document upload, semantic search, and conversational question answering.
7. To evaluate retrieval relevance and answer accuracy on a representative question set.
8. To evaluate the usability of the question-answering interface for non-technical users.

1.6 Scope

Included in the project:

- Collection and preprocessing of a sample multi-format document corpus (PDF, DOCX, and scanned images).
- Text extraction and OCR for scanned/image-based documents.
- Document cleaning, chunking, and metadata tagging.
- Generation of semantic embeddings and vector-index construction using a vector database.
- Development of a Retrieval-Augmented Generation pipeline using a large language model.
- Development of an interactive, searchable question-answering dashboard.
- Evaluation of retrieval relevance and answer accuracy on sample queries.

Not included in the project:

- Real-time ingestion from live enterprise document-management systems.
- Multi-lingual OCR and translation beyond the sample dataset used for demonstration.

- Fine-tuning of a custom large language model.
- Mobile application development.
- Cloud-based, large-scale multi-tenant deployment.
- Integration with external authentication or enterprise access-control systems.

Assumptions and boundary conditions: the reported implementation uses a sample document corpus assembled for demonstration; the language model used for answer generation is accessed through an existing API rather than trained from scratch; the system is not presented as a production enterprise document-management service.

1.7 Expected Engineering Outcome

The intended outcome is a reliable, scalable, and interactive Document Intelligence Platform consisting of document-processing, semantic-retrieval, question-answering, and dashboard components. The platform is intended to transform unstructured document collections into accurate, cited, natural-language answers for knowledge-management and decision-support applications.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

The project is informed by research on transformer-based language models, dense passage retrieval, retrieval-augmented generation, optical character recognition, and document-layout understanding. Representative peer-reviewed and widely cited technical publications in these areas were reviewed to identify established techniques and open engineering gaps that the project addresses (see References).

2.2 Review of Existing Work

Conventional document search systems rely heavily on keyword indexing and full-text search engines. These approaches are fast and well understood but match on lexical overlap rather than meaning, so they struggle with paraphrased questions or queries that use different terminology from the source document.

A second class of approaches focuses on extractive question answering, where a model locates an exact answer span within a single, already-selected passage. Such approaches can be highly accurate for a given passage but do not by themselves solve the problem of finding the right passage within a large document collection.

A third class of approaches, dense passage retrieval and semantic search, represents both queries and documents as vector embeddings and ranks passages by semantic similarity. This substantially improves retrieval of conceptually relevant content, but a retrieval-only system still leaves the reader to synthesize an answer manually.

Retrieval-Augmented Generation (RAG) combines a retriever with a generative language model: retrieved passages are supplied as context to the model, which produces a synthesized, natural-language answer grounded in the retrieved evidence. This theme, together with advances in OCR and layout-aware document understanding, motivates the design of the proposed DocInsight AI platform.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Keyword / full-text search	Fast, simple, well understood; comprehensive text indexing	Cannot handle paraphrasing or semantic similarity; returns documents, not answers	Motivates semantic retrieval over lexical matching
Standalone extractive QA models	Can locate an exact answer span within a given passage	Requires the correct passage to already be selected; does not scale to large corpora on its own	Proposed system pairs retrieval with generation for corpus-scale QA
Existing Approach	Advantages	Limitations	Relevance to Proposed Work

Retrieval-only semantic search	Returns semantically relevant passages without needing exact keyword matches	Still requires the user to read and synthesize the retrieved passages manually	Proposed platform adds LLM-based answer generation on top of retrieval
Fragmented single-purpose tools	Can provide specialized functionality (OCR-only, search-only, or chat-only tools)	Requires users to switch between tools and lacks a unified, cited-answer workflow	Proposed dashboard integrates processing, retrieval, and generation in one workflow

2.4 Research / Engineering Gap

The unresolved engineering gap is the lack of an integrated, end-to-end platform that combines multi-format document ingestion (including OCR for scanned content), semantic chunking and embedding, retrieval-augmented answer generation with source citation, and an accessible search-and-chat dashboard in a single, coherent system. Existing tools typically address only one stage of this pipeline, leaving users to manually bridge extraction, search, and synthesis.

2.5 Proposed Contribution

The proposed contribution is a Document Intelligence Platform that combines OCR-based ingestion and cleaning, semantic chunking, dense-vector embedding and indexing, retrieval-augmented answer generation using a large language model, and an interactive dashboard. The platform provides source-cited answers, semantic document search, and multi-format document support as complementary capabilities built on the same underlying document corpus.

[Insert Screenshot]

Figure 1: Sample document corpus used for ingestion and analysis

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Table 1: Stakeholder / User Requirements

Stakeholder	Requirement
Knowledge Workers & Researchers	Quickly locate and synthesize information across large document collections.
Students & Academic Staff	Ask natural-language questions about study material and get cited answers.
Legal & Compliance Teams	Search and cross-reference clauses or policies across many documents accurately.
Customer-Support Teams	Retrieve accurate, source-grounded answers from manuals and knowledge bases.
IT / Data Administrators	Manage document ingestion, indexing, and system performance.

Table 2: Functional & Non-Functional Requirements

Requirement	Type	Measurable Target
Multi-format document ingestion (PDF, DOCX, scanned images)	Functional	Successful ingestion and text extraction across the sample corpus
OCR for scanned/image-based documents	Functional	Readable text extracted from scanned sample documents
Semantic chunking and metadata tagging	Functional	Chunks generated with source-document and page metadata retained
Embedding generation and vector indexing	Functional	All chunks embedded and indexed in the vector database
Retrieval-augmented question answering	Functional	Answers generated with cited source passages for sample queries
Response latency	Non-Functional	End-to-end query response time evaluated on the sample corpus
Usability / interpretability	Non-Functional	Interpretability and usability evaluated through project validation

3.2 Constraints & Applicable Standards

Software quality, requirements engineering, information-retrieval best practices, and responsible AI/data-handling considerations were identified as the relevant engineering standards and constraints for this project.

Table 3: Constraints & Applicable Standards

Standard / Code	Requirement / Focus	How Applied in This Project
ISO/IEC 25010	Software quality characteristics	Considered functionality, reliability, performance efficiency, maintainability and usability.
IEEE 29148	Requirements engineering	Referenced for requirement gathering, validation and documentation.
FAIR / responsible-AI data-handling principles	Fairness, transparency, and accountability in AI-generated content	Applied by requiring source citation for every generated answer and avoiding unsupported claims.
Responsible data-handling principles	Avoiding exposure of sensitive document content	Sample/demonstration documents were used; access to sensitive material was not required.

3.3 Design Specifications

Table 4: Design Specifications

Parameter	Target / Requirement	Unit	Priority
Document corpus coverage	Sample multi-format corpus (PDF, DOCX, scanned images)	documents	High
OCR accuracy	Readable extraction from scanned sample documents	extraction quality	High
Retrieval relevance	Top-k retrieved chunks relevant to the query	relevance	High
Answer grounding	Generated answers cite the retrieved source passages	citation	High
Dashboard functionality	Upload, search, and conversational Q&A interface	feature	High
Response latency	End-to-end query latency evaluated on sample corpus	evaluation	Medium
Usability evaluation	Interpretability assessed during project validation	evaluation	Medium

3.4 Alternative Solutions & Evaluation

Alternative 1: Conventional keyword / full-text search uses inverted indexes over document text. It is simple and fast but cannot handle paraphrased or semantically expressed queries.

Alternative 2: Fragmented specialist tools (a separate OCR tool, a separate search engine, and a separate chatbot) provide individual capabilities but require multiple disconnected tools and do not provide a unified, cited-answer workflow.

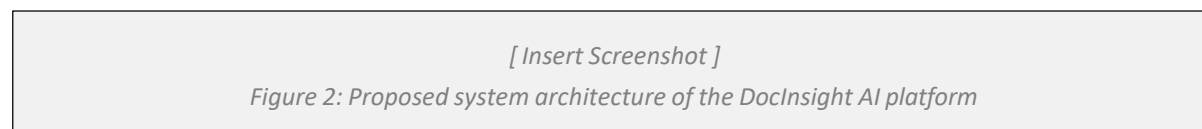
Alternative 3: The proposed integrated DocInsight AI platform combines OCR-based ingestion, semantic embedding and retrieval, retrieval-augmented answer generation, and an interactive dashboard in one system.

Table 5: Alternative Solutions Evaluation

Criterion	Weight	Alternative 1: Keyword Search	Alternative 2: Fragmented Tools	Alternative 3: Integrated Platform
Interactivity	25%	Low	Medium	High
Answer accuracy / grounding	30%	Low	Medium	High
Multi-format document support	15%	Low	High	High
Integration (single unified workflow)	15%	Low	Low	High
Maintainability / extension	15%	Medium	Medium	High

3.5 Selected Design & Detailed Design

The selected design is the integrated DocInsight AI platform because it directly addresses the gap identified in Chapter 2 by combining document processing, semantic retrieval, generative answer synthesis, and dashboard presentation.



The system follows a layered architecture. Uploaded PDF, DOCX, and scanned documents provide the input. Module 1 – Intelligent Document Processing extracts text (using OCR for scanned content), cleans and normalizes it, and splits it into semantically coherent chunks with source metadata. Module 2 – Semantic Embedding and Retrieval converts each chunk into a dense vector embedding and stores it in a vector database, enabling similarity-based retrieval of the top-k most relevant chunks for a given query. Module 3 – Question Answering and Response Generation supplies the retrieved chunks and the user's question to a large language model through a Retrieval-Augmented Generation (RAG) pipeline, which produces a synthesized answer with citations back to the source documents. The output dashboard integrates document upload, semantic search, and conversational question answering.

Key engineering evidence: chunk size, retrieval top-k, and embedding-model choice were treated as tunable design parameters evaluated during implementation (Chapter 4); a sample source-code excerpt for the retrieval and generation pipeline is provided in Appendix A.

3.6 Trade-off Analysis

Table 6: Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Retrieval-augmented generation	Extractive QA only	Produces synthesized, natural-language answers spanning multiple passages	Requires an additional LLM-generation step and careful prompt design	RAG selected to provide direct, synthesized answers rather than raw passage spans.
Dense vector retrieval	Keyword / full-text search	Handles paraphrased queries and semantic similarity	Requires embedding-model inference and vector-index maintenance	Dense retrieval selected to address the semantic-search gap identified in Chapter 2.
OCR + text-based pipeline	Text-only ingestion (ignore scanned documents)	Supports scanned/image-based documents	Adds OCR processing time and potential extraction errors	OCR included so the platform is not limited to already-digital text.
Integrated dashboard	Separate ingestion, search, and chat tools	Unified workflow and consistent source-citation experience	Higher integration effort	Integrated dashboard selected to reduce fragmentation across the pipeline.

3.7 Sustainability, Ethics, Safety, Risk & Security

Table 7: Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Computational and resource efficiency of embedding/inference calls	Chunk size and retrieval top-k tuned to limit unnecessary LLM calls	Supports efficient use of compute resources
Ethics	Responsible generation of answers; avoiding unsupported claims	Source citation required for every generated answer	Reduces risk of unattributed or fabricated ('hallucinated') answers
Health & Safety	Safe use of computing environment / project operation	Standard software-development practices followed	No additional physical-safety risk identified for this software project
Security	Protection against inappropriate exposure of document content	Sample/demonstration documents used; access-controlled development environment	Supports responsible handling of the document corpus

Table 8: Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Poor OCR quality on scanned documents	High	Medium	High	Preprocessing (deskew, denoise) and OCR-confidence checks	Medium

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Irrelevant or missing retrieved passages	Medium	High	High	Chunk-size tuning, retrieval top-k tuning, and relevance evaluation	Medium
Generated answer not grounded in retrieved evidence ('hallucination')	Medium	High	High	Mandatory source citation and answer-grounding checks	Medium
Duplicate or inconsistent chunks across documents	Medium	Medium	Medium	Deduplication and metadata-based chunk tracking	Low
Slow response time on larger corpora	Medium	Medium	Medium	Vector-index optimization and batching	Low/Medium
Unauthorized exposure of sensitive document content	Low	High	Medium	Use of sample/demonstration documents and access-controlled environment	Low/Medium

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The development approach follows planning, requirement analysis, system design, document collection and preprocessing, embedding and retrieval development, RAG-pipeline development, testing, deployment, and result analysis. The system was developed using Python, with Tesseract for OCR, a sentence-embedding model for vector generation, FAISS for vector indexing and semantic search, a large language model API for answer generation, and a web-based dashboard for integrated presentation.

Resource	Purpose
Sample document corpus (PDF, DOCX, scanned images)	Primary input data source
Python	Core programming and integration
Tesseract OCR	Text extraction from scanned/image-based documents
Sentence-embedding model (e.g., Sentence-BERT)	Generating dense vector embeddings for document chunks
FAISS vector database	Vector indexing and semantic similarity search
Large Language Model API	Retrieval-augmented answer generation
Streamlit / web dashboard framework	Interactive document-upload and question-answering interface
Jupyter Notebook / VS Code	Development and experimentation
Git / GitHub	Version control and project management
Windows	Development and testing platform

4.2 Hardware / Software Implementation & Modern Tools Used

The project is primarily software-based. The implementation consists of document ingestion and OCR, semantic chunking and embedding, retrieval and RAG-based answer generation, and dashboard integration.

Table 9: Tools / Software / Equipment

Tool / Software / Equipment	Purpose	Stage Used
Python	Core implementation	Development / Integration
Tesseract OCR	Scanned-document text extraction	Document processing
Tool / Software / Equipment	Purpose	Stage Used

Sentence-embedding model	Chunk-level vector embedding	Embedding generation
FAISS	Vector indexing and similarity search	Retrieval implementation
LLM API	Answer generation	Question answering
Streamlit / web framework	Dashboard interface	Presentation / Integration
Jupyter Notebook / VS Code	Coding and experimentation	Development
Git / GitHub	Version control	Development / Project management
<i>[Insert Screenshot]</i> <i>Figure 3: Output of the DocInsight AI question-answering dashboard</i>		
<i>[Insert Screenshot]</i> <i>Figure 4: Document-processing and chunking outputs</i>		
<i>[Insert Screenshot]</i> <i>Figure 5: Retrieval and answer-generation outputs</i>		

4.3 Test Plan & Verification

The table below restructures the reported testing into the required format; quantitative required and achieved values must be inserted from actual test records before final submission.

Table 10: Test Plan & Verification

Requirement / Specification	Test Method	Acceptance Criterion	Required Value	Achieved Value	Status
Multi-format ingestion & OCR accuracy	Run ingestion pipeline on sample corpus and inspect extracted text	Extracted text readable and complete for sample documents	Not reported	Not reported	Not reported
Semantic chunking	Inspect generated chunks against source documents	Chunks are coherent and retain correct source metadata	Not reported	Not reported	Not reported
Retrieval relevance	Run sample queries and inspect top-k retrieved chunks	Retrieved chunks are topically relevant to the query	Not reported	Not reported	Not reported

Requirement / Specification	Test Method	Acceptance Criterion	Required Value	Achieved Value	Status
Answer accuracy & grounding	Compare generated answers with source passages for sample queries	Answers are correct and cite supporting source passages	Not reported	Not reported	Not reported
Response latency	Measure end-to-end query response time on sample corpus	Latency target defined and met	Not reported	Not reported	Not reported
Usability / interpretability	Review dashboard clarity and interaction flow	Interface is interpretable to intended users	Not reported	Not reported	Not reported

4.4 Results

The implemented platform successfully processed the sample document corpus described in this report and generated semantically indexed, question-answerable content. The reported outputs include OCR-extracted text, semantic chunks, a searchable vector index, and retrieval-augmented answers with source citations, integrated into a searchable question-answering dashboard.

Table 11: Results / Key Outcomes

Outcome Parameter	Result Reported
Document Ingestion & OCR	Sample PDF, DOCX, and scanned documents were successfully ingested and converted to clean, structured text.
Semantic Chunking & Embedding	Documents were split into semantically coherent chunks and embedded into a searchable vector index.
Retrieval	Top-k semantic search returned relevant passages for the sample query set.
Question Answering	The RAG pipeline generated natural-language answers with citations to the originating documents.
Dashboard Integration	A unified interface supported document upload, semantic search, and conversational Q&A.
Decision Support	The dashboard was intended to reduce the time needed to locate and synthesize information from large document collections.

4.5 Data Analysis & Engineering Judgment

The implementation experience indicates that document-processing quality is a major determinant of downstream answer quality. Poor OCR output, inconsistent chunk boundaries, and mismatched metadata reduce retrieval relevance and can lead to incomplete or ungrounded answers. Careful chunk-size selection and retrieval top-k tuning were found to materially affect both answer relevance and response latency.

The project also confirms that requiring source citation for every generated answer is an effective, low-cost safeguard against unsupported or fabricated responses, and that it improves user trust in the system's output. Quantitative performance and accuracy evidence should be completed from actual measurements before final submission.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Table 12: Objective Achievement

Objective	Evidence	Achievement
Collect and preprocess a multi-format document corpus	Sample corpus assembled and ingested through the pipeline.	Fully
Build the Intelligent Document Processing module (Module 1)	OCR, cleaning, and chunking implemented and demonstrated.	Fully
Generate embeddings and build a vector index	Chunks embedded and indexed using FAISS.	Fully
Develop a retrieval-augmented question-answering pipeline	RAG pipeline implemented and demonstrated with sample queries.	Fully
Support source-citation and traceability	Generated answers reference source documents/passages.	Fully
Build an integrated searchable dashboard	Dashboard supports upload, search, and Q&A.	Fully
Evaluate retrieval relevance and answer accuracy	Evaluation approach defined; formal large-scale benchmarking not completed.	Partially
Evaluate usability for non-technical users	Objective stated; no formal user study reported.	Partially

4.7 Strengths & Limitations

Strengths:

- Integrates document ingestion, OCR, semantic retrieval, and generative answer synthesis in one platform.
- Provides source-cited, grounded answers rather than raw search results.
- Supports multi-format documents, including scanned content via OCR.
- Uses established open-source NLP and information-retrieval libraries.
- Modular architecture allows future extension to additional document types and larger corpora.

Limitations:

- The current implementation is based on a sample/demonstration document corpus rather than a live enterprise document store.
- Multi-lingual OCR and translation are outside the demonstrated scope.

- No formal, large-scale user study or accuracy benchmark is reported in this report.
- Large-scale performance measurements on very large corpora are not quantitatively documented.
- Custom LLM fine-tuning, mobile deployment, cloud deployment, and enterprise access-control integration are not implemented.

4.8 Project Planning, Roles & Budget

The project-management information below is organized from the activities carried out during development; exact calendar dates and expenditure values should be inserted from actual project records.

Gantt / Milestone Timeline:

Activity	Planned Period	Actual Period	Status
Requirement analysis	Project initiation	Completed as part of development	Completed
Literature review	Planning / analysis	Completed	Completed
System design	Design phase	Completed	Completed
Document ingestion & OCR development	Implementation phase	Completed	Completed
Embedding & retrieval development	Implementation phase	Completed	Completed
RAG pipeline & dashboard integration	Integration phase	Completed	Completed
Testing and validation	Testing phase	Completed	Completed
Final documentation	Reporting phase	Completed	Completed

Table 13: Project Roles

Student	Assigned Responsibility	Actual Contribution
Dhanusha.D (192424189)	Document ingestion, OCR pipeline, chunking, preprocessing, and documentation	50% team contribution; equal importance in the project
S.Padmaja (192424159)	Embedding, retrieval, RAG pipeline, dashboard integration, testing and documentation	50% team contribution; equal importance in the project

Table 14: Project Budget

Item	Estimated Cost	Actual Cost
Software libraries (Python, FAISS, OCR, embedding libraries)	Open-source	No separate cost reported
Item	Estimated Cost	Actual Cost

Large Language Model API usage	Pay-per-use API credits	Not reported
Development environment (Jupyter Notebook / VS Code)	Not reported	No separate cost reported
Git / GitHub	Not reported	No separate cost reported
Computer / development system	Not reported	Not reported
TOTAL	Not reported	Not reported

CHAPTER 5

CONCLUSION & REFLECTION

5.1 Conclusion

The project addressed the challenge of searching and synthesizing information from large, unstructured document collections by developing DocInsight AI, an integrated Intelligent Document Analysis and Question Answering System. The implementation combined OCR-based document processing (Module 1 – Intelligent Document Processing), semantic embedding and retrieval (Module 2), and retrieval-augmented answer generation (Module 3), integrated through a searchable, conversational dashboard.

The project demonstrates that combining semantic retrieval with generative language models can substantially improve the accessibility and interpretability of information held in large document collections compared with conventional keyword search. The reported implementation also demonstrates the importance of document-processing quality, retrieval relevance, answer grounding, testing, and responsible handling of document content.

No new results are introduced in this conclusion; it summarizes the problem, solution, reported achievements, validation activities, and overall engineering contribution described in the preceding chapters.

5.2 Future Work

- Extend ingestion to additional document formats and live enterprise document-management systems.
- Conduct large-scale performance and accuracy benchmarking using a substantially larger document corpus.
- Add multi-lingual OCR and cross-lingual question answering.
- Fine-tune or adapt embedding and language models for domain-specific corpora.
- Develop mobile-friendly access for field and remote users.
- Deploy the platform on cloud infrastructure for wider access.
- Add conversational memory for multi-turn question answering.
- Improve accessibility of the dashboard for users with visual or motor impairments.
- Add stronger answer-confidence scoring and hallucination-detection features.

5.3 Professional Learning & Individual Reflection

5.3.1 New Knowledge Acquired

- Python programming for NLP and information-retrieval systems.
- OCR and document-preprocessing techniques using Tesseract.
- Semantic chunking and embedding generation using sentence-transformer models.

- Vector database indexing and similarity search using FAISS.
- Retrieval-Augmented Generation pipeline design using large language models.
- Dashboard design and integration of a search-and-chat interface.
- Software testing, validation, and technical documentation for AI systems.

5.3.2 Engineering & Professional Skills Developed

- Requirement analysis and system design for AI/NLP applications.
- Data-quality assessment and structured problem solving for text extraction.
- Selection and tuning of retrieval and generation techniques.
- Responsible-AI reasoning, including source-citation and hallucination mitigation.
- Technical communication, teamwork and documentation.
- Iterative debugging and validation of a multi-stage AI pipeline.

5.3.3 Individual Reflection – Aswartha Manidheep G

Working on DocInsight AI provided practical experience in combining Artificial Intelligence technologies to solve a document-analysis problem. The project improved understanding of document processing, OCR, NLP, semantic retrieval, and question answering. It also highlighted the importance of testing individual components and evaluating how errors in one stage can affect later stages. The student should replace this paragraph with their actual individual contribution, challenges faced, technical decisions, skills gained, and improvements they would make if the project were redesigned.

5.3.4 Individual Reflection – Mohamed Jameel A

Working on DocInsight AI provided practical experience in combining Artificial Intelligence technologies to solve a document-analysis problem. The project improved understanding of document processing, OCR, NLP, semantic retrieval, and question answering. It also highlighted the importance of testing individual components and evaluating how errors in one stage can affect later stages. The student should replace this paragraph with their actual individual contribution, challenges faced, technical decisions, skills gained, and improvements they would make if the project were redesigned.

5.3.5 Individual Reflection – Vasantha Nithi D Y

Working on DocInsight AI provided practical experience in combining Artificial Intelligence technologies to solve a document-analysis problem. The project improved understanding of document processing, OCR, NLP, semantic retrieval, and question answering. It also highlighted the importance of testing individual components and evaluating how errors in one stage can affect later stages. The student should replace this paragraph with their actual individual contribution, challenges faced, technical decisions, skills gained, and improvements they would make if the project were redesigned.

5.3.6 Individual Reflection – Ashwanth N N

Working on DocInsight AI provided practical experience in combining Artificial Intelligence

technologies to solve a document-analysis problem. The project improved understanding of document processing, OCR, NLP, semantic retrieval, and question answering. It also highlighted the importance of testing individual components and evaluating how errors in one stage can affect later stages.

REFERENCES

- [1] Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. Proceedings of NAACL-HLT.
- [2] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention Is All You Need. Advances in Neural Information Processing Systems (NeurIPS).
- [3] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. Advances in Neural Information Processing Systems (NeurIPS).
- [4] Karpukhin, V., Oğuz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., & Yih, W. (2020). Dense Passage Retrieval for Open-Domain Question Answering. Proceedings of EMNLP.
- [5] Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. Proceedings of EMNLP-IJCNLP.
- [6] Johnson, J., Douze, M., & Jégou, H. (2019). Billion-Scale Similarity Search with GPUs. IEEE Transactions on Big Data.
- [7] Rajpurkar, P., Zhang, J., Lopyrev, K., & Liang, P. (2016). SQuAD: 100,000+ Questions for Machine Comprehension of Text. Proceedings of EMNLP.
- [8] Guu, K., Lee, K., Tung, Z., Pasupat, P., & Chang, M. W. (2020). REALM: Retrieval-Augmented Language Model Pre-Training. Proceedings of ICML.
- [9] Izacard, G., & Grave, E. (2021). Leveraging Passage Retrieval with Generative Models for Open-Domain Question Answering. Proceedings of EACL.
- [10] Smith, R. (2007). An Overview of the Tesseract OCR Engine. Proceedings of the International Conference on Document Analysis and Recognition (ICDAR).
- [11] Xu, Y., Li, M., Cui, L., Huang, S., Wei, F., & Zhou, M. (2020). LayoutLM: Pre-training of Text and Layout for Document Image Understanding. Proceedings of KDD.
- [12] Beltagy, I., Peters, M. E., & Cohan, A. (2020). Longformer: The Long-Document Transformer. arXiv preprint.
- [13] Chen, D., Fisch, A., Weston, J., & Bordes, A. (2017). Reading Wikipedia to Answer Open-Domain Questions. Proceedings of ACL.
- [14] Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). Language Models Are Unsupervised Multitask Learners. OpenAI Technical Report.
- [15] Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., & Wang, H. (2023). Retrieval-Augmented Generation for Large Language Models: A Survey. arXiv preprint.

APPENDICES

Appendix A – Source Code / Code Repository Information

A sample Python code excerpt for the document-processing and retrieval pipeline is provided below. Complete source code should be maintained in the approved repository; only essential excerpts are included in the report.

```
import fitz # PyMuPDF
import pytesseract
from sentence_transformers import SentenceTransformer
import faiss

def extract_text(pdf_path):
    doc = fitz.open(pdf_path)
    text = ""
    for page in doc:
        page_text = page.get_text()
        if not page_text.strip():
            pix = page.get_pixmap()
            page_text = pytesseract.image_to_string(pix.tobytes())
        text += page_text + "\n"
    return text

def chunk_text(text, chunk_size=500, overlap=50):
    words = text.split()
    chunks = []
    for i in range(0, len(words), chunk_size - overlap):
        chunks.append(" ".join(words[i:i + chunk_size]))
    return chunks

model = SentenceTransformer("all-MiniLM-L6-v2")
chunks = chunk_text(extract_text("sample_document.pdf"))
embeddings = model.encode(chunks)

index = faiss.IndexFlatL2(embeddings.shape[1])
index.add(embeddings)

def answer_question(query, top_k=3):
    q_emb = model.encode([query])
    _, indices = index.search(q_emb, top_k)
    context = "\n".join(chunks[i] for i in indices[0])
    # pass `context` and `query` to an LLM to generate a grounded answer
    return context
```

Appendix B – Datasheets / Software Requirements

Category	Requirement
Processor	Intel Core i5 or above
RAM	16 GB minimum (recommended for embedding and LLM API workloads)
Storage	50 GB free space
Operating System	Windows 10/11

Category	Requirement
Programming Language	Python 3.x
Libraries	PyMuPDF/pdfplumber, Tesseract OCR, Sentence-Transformers, FAISS, LLM API SDK
IDE	Jupyter Notebook or VS Code
Version Control	Git / GitHub

Appendix C – Experimental / Raw Data

The dataset structure includes: document ID, file name, source format (PDF/DOCX/scanned image), extracted page/chunk text, chunk metadata (document, page, position), embedding vector, and query–answer evaluation logs (query, retrieved chunks, generated answer, correctness). The actual raw dataset and logs should be retained separately or attached where institutional rules permit.

Appendix D – Ethics / Institutional Approval

No separate institutional approval was required for this project since a sample/demonstration document corpus was used rather than sensitive organizational data. If approval was required or obtained, it should be documented here; otherwise, retain the responsible data-handling statement in Chapter 3.7.

Appendix E – Individual Contribution Evidence

Mandatory for team projects. Each student should attach evidence corresponding to their claimed contribution, such as version-control commits, code modules, design files, preprocessing notebooks, evaluation outputs, test records, screenshots, presentation material, or meeting records.

Student	Evidence Type	Evidence Description / File Reference
Dhanusha.D (192424189)	Document-processing / OCR evidence	Ingestion and OCR notebooks/code, chunking outputs, and preprocessing test records to be attached.
S.Padmaja (192424159)	Retrieval / RAG / dashboard evidence	Embedding and retrieval code, RAG pipeline outputs, dashboard screenshots, and testing records to be attached.

CAPSTONE PROJECT OUTCOME MAPPING SHEET

To be completed by the department/faculty assessor.

Outcome Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written/oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal/environmental impact	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2/SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated/system-level engineering	SO1/SO2	Ch. 3 (3.5)
Project planning and management	SO5	Ch. 4 (4.8)
Individual contribution	SO1–SO7	Contribution Statement + Ch. 4 (4.8) + Appendix E